GPU-PARALLELISED ORBIT PROPAGATION TOOL FOR DETERMINING THE IMPACT OF PROPULSION AND POINTING RELIABILITY ON PW-SAT3'S COLLISION AVOIDANCE CAPABILITIES

Michał P. Kiedrzyński[(1)], Igor Januszkiewicz[(1)], Marek Polewski[(1)], Kamil Koszarski[(1)], and Krzysztof Kaczmarski[(2)]

[(1)]*Students' Space Association, Warsaw University of Technology, Nowowiejska 21/25, 00-665 Warsaw, Poland. email: ska.satellites@rkn.pw.edu.pl*
[(2)]*Faculty of Mathematics and Information Science, Warsaw University of Technology, ul. Koszykowa 75, 00-662 Warsaw, Poland. email: krzysztof.kaczmarski@pw.edu.pl*

Abstract

This paper presents the Open GPU-Parallelised Orbit Propagator (OpenGPOP), which utilises the computational power and parallel processing capabilities of Graphics Processing Units (GPUs) to perform high-fidelity orbit propagation of many objects simultaneously. In the conducted tests, OpenGPOP outperformed traditional CPU-based propagators on tasks involving more than 10 satellites and, for large batches (10,000), achieved a 580x speed-up. The paper provides a detailed description of the implemented numerical methods, force model, conjunction checks, and parallelisation techniques. The resulting source code is also given.

The tool is subsequently applied to the case of the PW-Sat3 satellite, a student project equipped with an Attitude and Orbit Control System (AOCS) featuring warm-gas propulsion, developed by a student team. The study investigates the impact of AOCS inaccuracies on the satellite's collision avoidance capabilities using Monte Carlo simulations. The results demonstrate that OpenGPOP can provide valuable insights into the performance of collision avoidance manoeuvres under realistic conditions.

1 INTRODUCTION

PW-Sat is a family of student-made CubeSats developed by the Students' Space Association at the Warsaw University of Technology. Apart from extensive educational value, the overarching theme of these missions is testing the effectiveness of various deorbitation techniques and proving their viability even in highly budget-constrained student projects. PW-Sat (1U, 2012-2014) and PW-Sat2 (2U, 2018-2021) satellites tested two passive methods: a deorbitation tail and a deorbitation sail, respectively.

Unlike its predecessors, the newest member of the family, PW-Sat3 (3U), will utilise an active deorbitation system based on a self-developed warm-gas propulsion. Additionally, a portion of the propellant is set to be used for other orbital manoeuvres, such as collision avoidance. The propulsion subsystem is a part of a broader, custom Attitude and Orbit Control System (AOCS), similarly designed and implemented by the student team.

Due to the nature of the project, the team had to rely on relatively simple and cost-effective components for the AOCS, utilising only Sun sensors and magnetometers for attitude determination. As a

result, the pointing accuracy and consistency of the AOCS are limited, especially compared to systems utilising star trackers, which often provide sub-arc-minute accuracy. Additional inaccuracies are introduced by the propulsion system.

In light of these limitations, the team decided to investigate whether the proposed attitude control and propulsion systems would provide sufficient collision avoidance capabilities. For the analysis, a Monte Carlo simulation was chosen, as it provides a quantitative estimation and is relatively easy to implement and interpret. First, a series of random boundary conditions, based on assumed error margins, is generated, followed by precise numerical propagation of each resulting orbit. Then, the distance of closest approach is computed, determining the manoeuvre's effectiveness.

However, to obtain satisfactory results, the method requires a large number of satellite instances, necessitating considerable computational power. Graphics Processing Units (GPUs) are well-suited for Monte Carlo simulations due to their ability to handle massive parallel computations efficiently. These methods rely on running many independent random simulations, making them inherently parallelisable. Unlike Central Processing Units (CPUs), which are primarily optimised for processing a few tasks sequentially as fast as possible, GPUs trade lower single-thread performance for the ability to execute thousands of threads in parallel.

These factors inspired the development of the Open GPU-Parallelised Orbit Propagator (OpenGPOP), capable of numerically propagating large numbers of orbits in parallel. The tool takes a set of initial conditions – positions and velocities – for each satellite and propagates them for a specified duration, outputting the state vectors at the end of this period. It is also capable of conjunction checking, by calculating the distance of closest approach between objects and returning it to the user. The solution utilises the Compute Unified Device Architecture (CUDA) platform, developed by NVIDIA, which is a widely adopted standard solution for scientific and high-performance computing. In pursuit of general accessibility of scientific tools, OpenGPOP is an open-source project available publicly under the GNU General Public License v3.0. The repository URL is provided in Appendix A.

The main goals of this paper are to asses whether the developed tool can provide accurate results with high performance, and to apply it to the PW-Sat3 use case, providing insights into the satellite's collision avoidance capabilities. In the subsequent sections of the paper, the tool is described in detail, including the implemented numerical methods, force model, reference system transformations, and conjunction checks. The performance and accuracy of the solution are examined and compared with other existing solutions. The tool is then applied to the aforementioned PW-Sat3 use case, and the resulting analysis is provided. Finally, the wider applicability of OpenGPOP is discussed, including satellite swarms and fragmentation events.

2 METHODOLOGY

This section describes how OpenGPOP predicts satellite states in time, how accelerations are computed from the force model, and how closest approaches between objects are detected. The numerical integrator is presented first, followed by the gravity model and conjunction checks.

2.1 Integrator

Orbit propagation is cast as a second-order initial value problem,

$$\ddot{\mathbf{r}} = \mathbf{a}(t, \mathbf{r}, \dot{\mathbf{r}}) \,, \tag{1}$$

where $\mathbf{r}$ is position in an inertial frame and $\mathbf{a}$ is acceleration from the force model (subsection 2.2). OpenGPOP predicts future states with a fixed time step using the Gauss–Jackson method [1]. This is

a multi-step predictor–corrector scheme widely used for astrodynamics: it predicts the next position and velocity from a history of accelerations at previous steps, evaluates the force at the predicted state, and then applies a corrector to refine the result. The formulation and coefficient tables follow Berry and Healy [1], and the implementation uses eighth-order formulas in ordinate form.

Multi-step methods need several past samples before the main step can run. OpenGPOP therefore starts each trajectory with an eighth-order Runge–Kutta integrator to build the early states and acceleration history, then switches to the Gauss–Jackson cycle. Each normal step alternates prediction and correction; the corrector can be applied more than once per step so that the acceleration used in the history matches the refined state more closely.

The same formulas are applied independently to every propagated object. On the GPU, acceleration histories and state vectors for all trajectories reside in device memory. Matrix–vector products that combine past accelerations with the fixed Gauss–Jackson and summed-Adams coefficient tables are performed with batched linear algebra functions from the cuBLAS [2] library, and other updates to positions, velocities, and histories are implemented as custom CUDA kernels. This layout matches Monte Carlo use cases, where many orbits differ only in initial conditions, while sharing the same time points and integration logic.

2.1.1 Runge–Kutta startup as batched linear algebra

The startup advances four steps forward and four backwards from the epoch with an eighth-order Runge–Kutta method. Each step consists of ten stages, calculating accelerations in intermediate states. Those accelerations are stored as columns of a 3×10 matrix for each satellite. This layout allows calculating the next intermediate steps using single matrix–vector products with the Butcher table coefficients, which are held as columns of a transposed coefficient matrix on the device. The final part of the Runge–Kutta step is a weighted sum of the ten stages, which is implemented as a matrix–vector product.

After the forward and backward steps are complete, the history of accelerations for the main Gauss–Jackson cycle is built by calculating the acceleration at each point. Then, the mid-corrector formula is applied to correct the initial states. The formula is implemented according to the same pattern of matrix–vector products with the coefficient tables, followed by custom kernels to finalise the correction. The corrector is applied multiple times.

2.1.2 Gauss–Jackson steps as batched linear algebra

After startup, the main cycle of Gauss–Jackson steps begins. Each step starts with a predictor, which calculates the next position and velocity from the history of accelerations using matrix–vector products with the coefficient tables. The predicted state is then used to evaluate the acceleration at the next time point.

The corrector precomputes partial sums over the first eight acceleration columns with two more matrix–vector multiplications. Inside each corrector iteration, the new state is calculated using partial sums and the new acceleration, and then the acceleration history is updated with the new value. The corrector can be applied multiple times to further refine the state.

2.2 Force model

In its current form, the force model implemented in OpenGPOP uses a single acceleration source – the Earth's gravitational potential, modelled using spherical harmonics. Equation 2 presents the

conventional representation of this potential using a series expansion in terms of surface spherical harmonics [3].

$$U = \frac{\mu}{r} + \sum_{n=2}^{N} \sum_{m=0}^{n} \frac{\mu}{r} \left(\frac{R}{r}\right)^n P_{n,m}(\epsilon) \left(C_{n,m} \cos(m\lambda) + S_{n,m} \sin(m\lambda)\right) \tag{2}$$

where:

- μ – Earth's gravitational constant (GM)
- R – Earth's equatorial radius
- r – magnitude of the position vector $\mathbf{x} = (x_1, x_2, x_3)$
- ϵ – sine of the geodetic latitude $\epsilon = x_3/r$
- $P_{n,m}(\epsilon)$ – associated Legendre functions
- $C_{n,m}, S_{n,m}$ – harmonic coefficients
- λ – geodetic longitude
- N – maximum degree of the expansion

The corresponding acceleration a is expressed as

$$a = \nabla U \tag{3}$$

The implemented computation of this acceleration follows an algorithm published by Robert G. Gottlieb in 1993 [3], with minimal modifications to leverage the tool's parallel nature. It is divided into six main phases, described below.

2.2.1 Phase 1 – Initialisation

Upon initialisation, OpenGPOP loads a chosen gravity model from a `.gfc` file. It accepts any static, time-invariant sets of normalised coefficients expressed in the International Centre for Global Earth Models (ICGEM) format. Other datasets, such as leap-second data, nutation model coefficients, and Earth Orientation Parameters (EOP), are also loaded from their corresponding files. Additionally, in this stage, all state-invariant auxiliary values are calculated and stored to optimise subsequent computations.

2.2.2 Phase 2 – Reference system transformation

Since the input position and velocity vectors are expressed in the Geocentric Celestial Reference System (GCRS), they must first be transformed to the International Terrestrial Reference System (ITRS) before Eqs. 2-3 can be applied. This is achieved by creating a transformation matrix $M(t)$, and multiplying the vectors by it (Eq. 4).

$$\mathbf{x}_{\text{[ITRS]}} = M(t)\mathbf{x}_{\text{[GCRS]}} \tag{4}$$

In regular, CPU-focused implementations, the creation of this matrix is usually done right before the multiplication, and discarded afterwards. However, since in OpenGPOP accelerations for all entities are computed at predefined time points, pre-computing and storing the matrices yields substantial savings in execution time. Moreover, this approach allows for full utilisation of the GPU architecture, as $M(t)$ matrices can be computed in parallel, using one thread per time point.

The implementation uses the equinox-based coordinate transformation, consistent with the International Astronomical Union (IAU) 2000 / 2006 resolutions as described in the IERS Conventions 2010 [4]. For precession and nutation, the four-rotation Fukushima-Williams method is used. Earth's rotation is calculated using Greenwich Apparent Sidereal Time (GAST) and the Equation of the Equinoxes. Finally, polar motion is accounted for using polar coordinates x_p, y_p, linearly interpolated from daily EOP, and the TIO locator s'. All the involved matrix multiplications were performed using the GPU-accelerated cuBLAS library.

Since $M(t)$ is a composition of several rotation matrices, which are naturally orthogonal, $M(t)$ is also orthogonal, providing a convenient property (Eq. 5).

$$M^{-1}(t) = M^T(t) \quad \wedge \quad \mathbf{x}_{\text{[ITRS]}} = M(t)\mathbf{x}_{\text{[GCRS]}} \quad \Rightarrow \quad \mathbf{x}_{\text{[GCRS]}} = M^T(t)\mathbf{x}_{\text{[ITRS]}} \tag{5}$$

2.2.3 Phase 3 – Computation of normalised derived Legendre polynomials

Gottlieb's algorithm [3] does not operate directly on associated Legendre functions $P_{n,m}$, but rather uses the so-called derived Legendre polynomials P_n^m, as defined in Eq. 6.

$$P_n^m = \frac{r^m}{\rho^m} P_{n,m} = \frac{\partial^m P_n}{\partial \epsilon^m} \tag{6}$$

where

- $\rho^2 = x_1^2 + x_2^2$
- P_n are the Legendre polynomials

In practice, since the harmonic coefficients $C_{n,m}, S_{n,m}$ are most often provided in their normalised forms $\overline{C_{n,m}}, \overline{S_{n,m}}$, the derived Legendre polynomials are also normalised ($\overline{P_n^m}$). The normalised derived Legendre polynomials can be efficiently calculated in parallel, using recursive relations from Eq. 7.

$$\begin{aligned} \overline{P_0^0} &= 1 & \overline{P_n^{n-1}} &= \epsilon\sqrt{2n+1}\,\overline{P_{n-1}^{n-1}} \\ \overline{P_n^n} &= \sqrt{\frac{2n+1}{2n}}\,\overline{P_{n-1}^{n-1}} & \overline{P_n^m} &= \xi(n,m)\epsilon\,\overline{P_{n-1}^m} - \eta(n,m)\,\overline{P_{n-2}^m} \end{aligned} \tag{7}$$

where $\xi(n,m), \eta(n,m)$ are coefficient functions pre-computed during the initialisation phase (2.2.1). OpenGPOP leverages the aforementioned properties, running m threads for every propagated entity, achieving a substantial speed-up in computations.

2.2.4 Phase 4 – Sines and cosines calculation

Similarly to derived Legendre polynomials, sines and cosines from Eq. 2 can also be calculated using recursive methods. Gottlieb [3] defines

$$\begin{aligned}\bar{C}_m &= \frac{\rho^m}{r^m}\cos(m\lambda) \\ \bar{S}_m &= \frac{\rho^m}{r^m}\sin(m\lambda)\end{aligned} \tag{8}$$

It is now possible to calculate $\bar{C}_m, \bar{S}_m$ using the property in Eq. 9, which OpenGPOP does, running one thread per entity.

$$\begin{aligned}\bar{C}_1 &= \frac{x_1}{r} \qquad \bar{C}_m = \bar{C}_1\bar{C}_{m-1} - \bar{S}_1\bar{S}_{m-1} \\ \bar{S}_1 &= \frac{x_2}{r} \qquad \bar{S}_m = \bar{S}_1\bar{C}_{m-1} + \bar{C}_1\bar{S}_{m-1}\end{aligned} \tag{9}$$

2.2.5 Phase 5 – Element summation

Further following Gottlieb [3], let us define

$$\begin{aligned}\Gamma &= \sum_{n=2}^{N}\left(\frac{R}{r}\right)^n \Gamma_n \qquad \Gamma_n = \overline{C_{n,0}}(n+1)\overline{P_n^0} + \sum_{m=1}^{n}(1+n+m)\overline{P_n^m}(\overline{C_{n,m}}\bar{C}_m + \overline{S_{n,m}}\bar{S}_m) \\ J &= \sum_{n=2}^{N}\left(\frac{R}{r}\right)^n J_n \qquad J_n = \sum_{m=1}^{n} m\overline{P_n^m}(\overline{C_{n,m}}\bar{C}_{m-1} + \overline{S_{n,m}}\bar{S}_{m-1}) \\ K &= \sum_{n=2}^{N}\left(\frac{R}{r}\right)^n K_n \qquad K_n = -\sum_{m=1}^{n} m\overline{P_n^m}(\overline{C_{n,m}}\bar{S}_{m-1} - \overline{S_{n,m}}\bar{C}_{m-1}) \\ H &= \sum_{n=2}^{N}\left(\frac{R}{r}\right)^n H_n \qquad H_n = \zeta(n,0)\overline{C_{n,0}}\overline{P_n^1} + \sum_{m=1}^{n}\zeta(n,m)\overline{P_n^{m+1}}(\overline{C_{n,m}}\bar{C}_m + \overline{S_{n,m}}\bar{S}_m)\end{aligned} \tag{10}$$

where $\zeta(n,m)$ is a coefficient function pre-computed during the initialisation phase (2.2.1).

Here, the computations are performed in four steps. First, using $N(N-1)/2$ threads per entity, all possible n, m combinations for the under-sum values in Γ_n, J_n, K_n, H_n are calculated and saved to a single, large array of size $N(N-1)/2$. Next, using a GPU-accelerated reduction algorithm from the Thrust [5] library and a custom iterator, this array is reduced to size N, by summing up all n, m combinations with the same n. This effectively calculates the values of Γ_n, J_n, K_n, H_n. Then, each of the values in the N-sized array is multiplied by the corresponding $(R/r)^n$ values. Finally, the array is reduced again using the Thrust library, producing Γ, J, K, H.

2.2.6 Phase 6 – Acceleration calculation

The last step involves computing Eq. 11, using one thread per entity.

$$a = -\frac{\mu}{r^2}\left(\left(\Gamma + 1 + \frac{x_3 H}{R}\right)\frac{\mathbf{x}}{r} - \begin{bmatrix}J\\K\\H\end{bmatrix}\right) \tag{11}$$

2.2.7 Accommodating other acceleration sources

Although only the harmonic gravitational potential is currently taken into account, the modular, object-oriented design of the force model allows for easy addition of other acceleration sources in the future. When multiple sources are present, OpenGPOP calculates and saves each source to a column of a temporary matrix, then multiplies a ones vector by that matrix to sum the vectors. This is done using the cuBLAS library.

2.3 Conjunction checks

OpenGPOP features conjunction checking capabilities by calculating the closest approaches between predefined satellite pairs. The user can group the propagated entities into several sets, where each set follows three rules:

1. entities within the same set are not checked against one another,
2. all pairs of entities not belonging to the same set are checked,
3. the sets must be disjoint.

First, all entities are propagated for a defined number of steps and with a chosen step size. After each step, all relevant pairs are tested to see if the current separating distance is the shortest so far. If so, the position, velocity, and acceleration vectors at that point (x_k, v_k, a_k), as well as one step before $(x_{k-1}, v_{k-1}, a_{k-1})$ and one after $(x_{k+1}, v_{k+1}, a_{k+1})$, are saved.

Then, the path between points $k-1$, k, and $k+1$ is interpolated using 8th degree Hermite interpolation polynomials. This ensures that the positions and their derivatives across all three nodes are matched exactly and provides an accurate approximation of the path between the nodes.

Finally, to find the exact moment of closest approach, a root of the dot product of relative position and velocity (Eq. 12), is numerically found using the secant method. Knowing the time t of closest approach, the calculation of the minimal separating distance is straightforward using the interpolation polynomials.

$$f(t) = \begin{bmatrix} p_{A1}(t) - p_{B1}(t) \\ p_{A2}(t) - p_{B2}(t) \\ p_{A3}(t) - p_{B3}(t) \end{bmatrix} \cdot \begin{bmatrix} \dot{p}_{A1}(t) - \dot{p}_{B1}(t) \\ \dot{p}_{A2}(t) - \dot{p}_{B2}(t) \\ \dot{p}_{A3}(t) - \dot{p}_{B3}(t) \end{bmatrix} = 0 \tag{12}$$

where:

- $p_{Ai}(t)$ – interpolation polynomial of the first entity in the i coordinate.
- $p_{Bi}(t)$ – interpolation polynomial of the second entity in the i coordinate.
- $\dot{p}_{Ai}(t)$ – derivative of the interpolation polynomial of the first entity in the i coordinate.
- $\dot{p}_{Bi}(t)$ – derivative of the interpolation polynomial of the second entity in the i coordinate.

3 EVALUATION

3.1 Force model tests

The correctness of the reference system transformations, described in 2.2.2, was tested against the International Astronomical Union's (IAU) Standards of Fundamental Astronomy (SOFA) ANSI C library [6]. The functions implemented in OpenGPOP were found to return identical or nearly identical results (differing by at most 10^{-11}) for the same inputs.

The accuracy of the gravitational acceleration calculation was checked against a MATLAB implementation of Gottlieb's algorithm, published in Eckman et al. 2016 [7]. In the conducted tests, OpenGPOP returned the accelerations with a precision of at least $10^{-14}\,\mathrm{m/s^2}$.

3.2 Precision

The integrator's correctness was verified through tests on polynomial functions. As it is a 8th-order method, the integrator should be able to propagate polynomials of degree 8 or less with zero error. The tests confirmed that the integrator achieves this, with errors on the order of machine precision.

The orbit propagation precision of OpenGPOP was evaluated by comparison against the precise orbit ephemerides (POE) of the Sentinel-1A satellite [8]. A 24-hour propagation was performed starting from an initial state vector extracted from the POE file. The resulting position error, calculated as the Euclidean distance between the propagated position and the reference POE position at each time step, is shown in Figure 1.

For validation, the results were compared with the Orekit library, a well-established open-source space dynamics library written in Java [9]. Two Orekit configurations were tested: a numerical propagator with a 4th-order Runge-Kutta integrator (RK4), and a numerical propagator with a Dormand-Prince 8(5,3) integrator (DP853). Both of these configurations use a single source force model based on spherical harmonics with degree 70 and order 70. Figure 1 presents the results of this analysis.

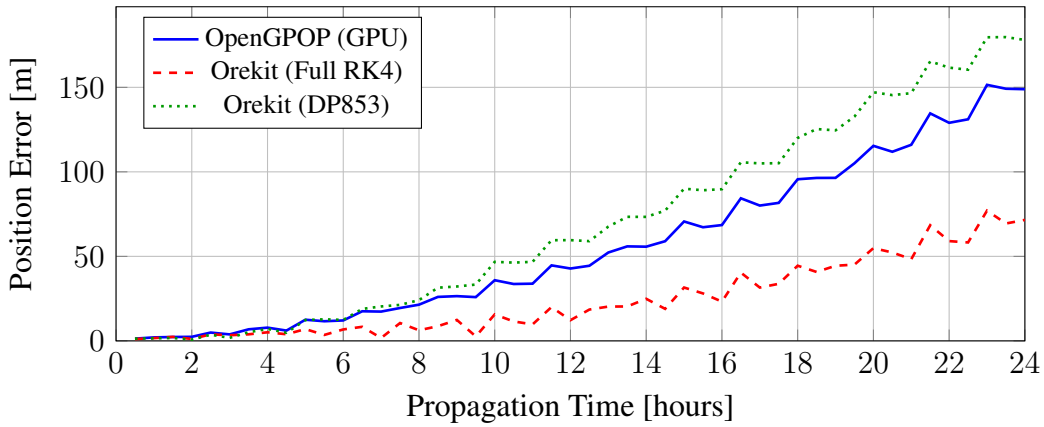


Figure 1. Position error over a 24-hour propagation period compared to Sentinel-1A precise ephemerides.

3.3 Performance

To assess the computational efficiency of OpenGPOP, a performance benchmark was conducted by propagating a varying number of satellites for a fixed duration of 6 hours. The test measured the total

execution time required to propagate N satellites, where N ranged from 1 to 10,000. For comparison, the same scenario was executed using propagators from section 3.2.

The benchmark was performed on the Eden cluster at the Faculty of Mathematics and Information Science at Warsaw University of Technology, which utilises an NVIDIA H100 GPU and an AMD EPYC 9534 CPU.

As shown in Figure 2, OpenGPOP demonstrates a significant advantage in scalability. Due to the massive parallelism of the GPU implementation, the execution time for OpenGPOP remains nearly constant as the number of satellites increases from 1 to 1000. In contrast, the CPU-based Orekit propagators exhibit a linear increase in execution time. For $N = 10{,}000$ satellites, OpenGPOP is approximately 580 times faster than the Orekit Numerical Propagator (RK4). Even compared to the variable-step Dormand-Prince integrator in Orekit, OpenGPOP maintains a substantial performance lead for large batched propagations.

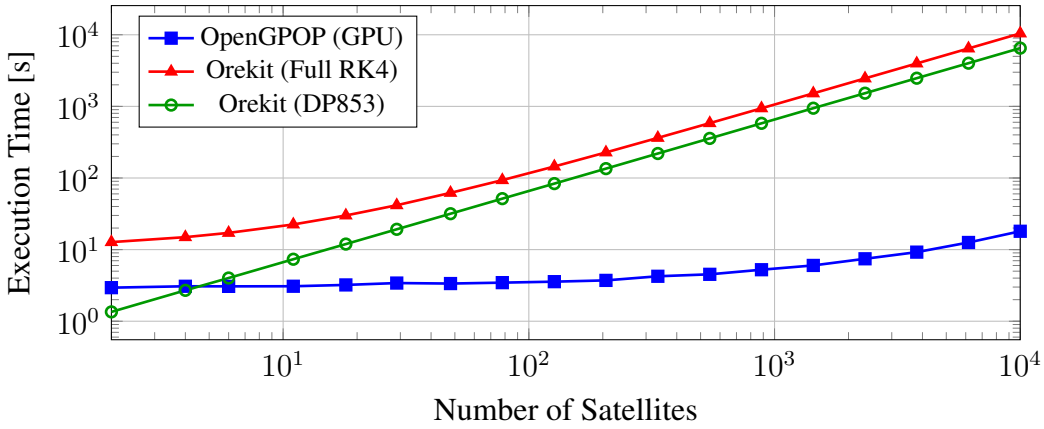


Figure 2. Execution time for propagating N satellites for 6 hours (Log-Log scale).

4 APPLICATION TO PW-SAT3

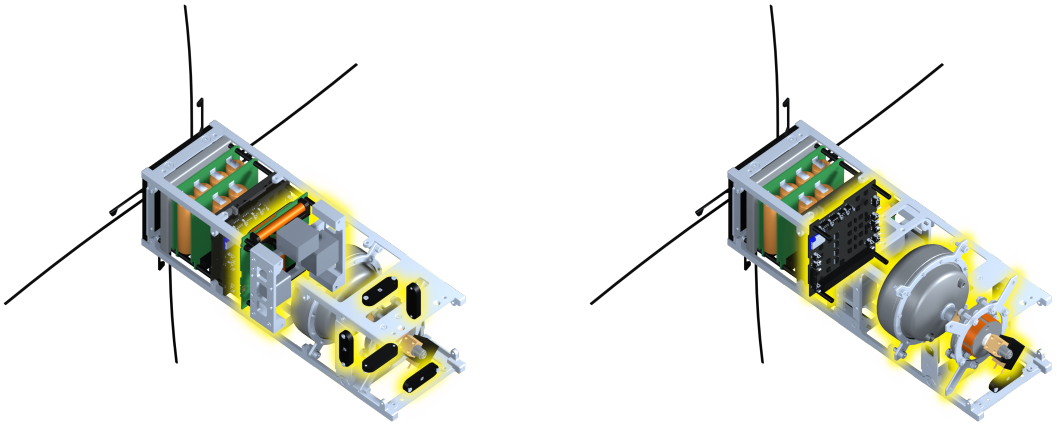
The primary application of the presented tool is the analysis of the impact of inaccuracies in the AOCS of the PW-Sat3 satellite on the consistency of collision avoidance capabilities. These inaccuracies come from two primary sources: the attitude control subsystem (Fig. 3a) and the propulsion subsystem (Fig. 3b).

Attitude control imprecision stems from uncertainties in attitude determination. Due to financial constraints, the team could not afford highly accurate sensors, such as star trackers, and had to rely solely on fine sun sensors and magnetometers. This, in turn, leads to non-negligible variations in the direction of the applied thrust during manoeuvres, which can cause inconsistencies.

Additional inaccuracies result from the propulsion system performance. These are caused by uncertainties in the propellant heating conditions in the heating chamber, which can lead to variations in thrust magnitude. Furthermore, the direction of the applied thrust can be affected by misalignments of the thruster during satellite assembly.

4.1 Inaccuracies model

The inaccuracies model is divided into two parts, corresponding to uncertainties in the direction and the magnitude of the applied thrust.



(a) Attitude determination and control subsystem (b) Warm-gas propulsion subsystem

Figure 3. Key AOCS subsystems of the PW-Sat3 satellite, relevant to the analysis of collision avoidance capabilities

The direction uncertainty is modelled as a Gaussian distribution of the pointing error, which is the angle between the intended and actual direction of the applied thrust, in both pitch and yaw directions in the LVLH frame. The mean of the distribution is assumed to be 0, as no systematic bias in the pointing error is expected. Several different values of the standard deviation are tested to examine how the consistency of collision avoidance capabilities changes with increasing pointing error.

Similarly, the magnitude uncertainty is modelled as a Gaussian distribution of the ΔV magnitude applied during the manoeuvre. The mean of the distribution is set to $2\,\mathrm{cm/s}$, which is the nominal ΔV value of one burn. Different values of the standard deviation are tested, as is the case with the pointing error.

4.2 Scenarios and methodology

In the conducted analysis, the PW-Sat3 satellite is placed in a circular orbit above the equator, on the X-axis of the ECI frame, at an altitude of 500 km. It is assumed to be performing a collision avoidance manoeuvre in response to a predicted conjunction with another object. The other object is placed in the same plane, on the opposite side of the X-axis, and at the same altitude. However, it follows an elliptical orbit with a period of 11 hours and 53 minutes, moving in the opposite direction. The predicted conjunction is set to occur 11 hours and 49 minutes after the manoeuvre, with the distance of closest approach of $70\,\mathrm{cm}$. The satellite performs a single, impulsive retrograde burn to mitigate the collision risk.

For the Monte Carlo analysis, the pointing error and ΔV magnitude are sampled independently of their respective distributions and applied to the nominal manoeuvre. The resulting satellite instances are placed in the same initial position as the nominal case, but with the velocity vector modified according to the sampled variables. The instances and the other object are placed in two separate sets in OpenGPOP, so that conjunction checks are performed between each instance and the object, but not between the instances themselves. The trajectories of these states and the object are then propagated for 12 hours, and the distance of closest approach is calculated for each instance.

The direction and magnitude variations are first tested separately to examine their individual impacts on the consistency of collision avoidance capabilities. Then, a combined scenario is tested to examine their joint impact.

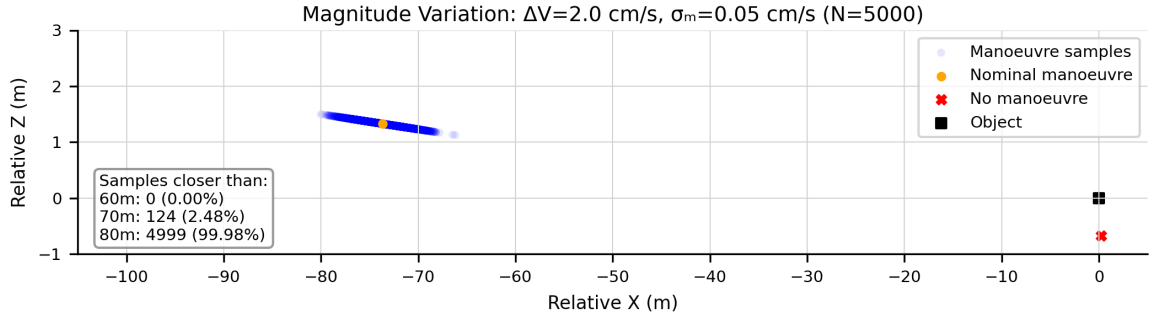
4.3 Results

The following results were generated using 5000 random samples for each scenario. In all scenarios, the resulting relative position between the object and the samples in the Y direction was negligible; therefore, for legibility, the figures present only the XZ-plane projection of the relative positions at the times of closest approach. In the nominal case without inaccuracy modelling, the distance of closest approach was approximately $74\,\mathrm{m}$.

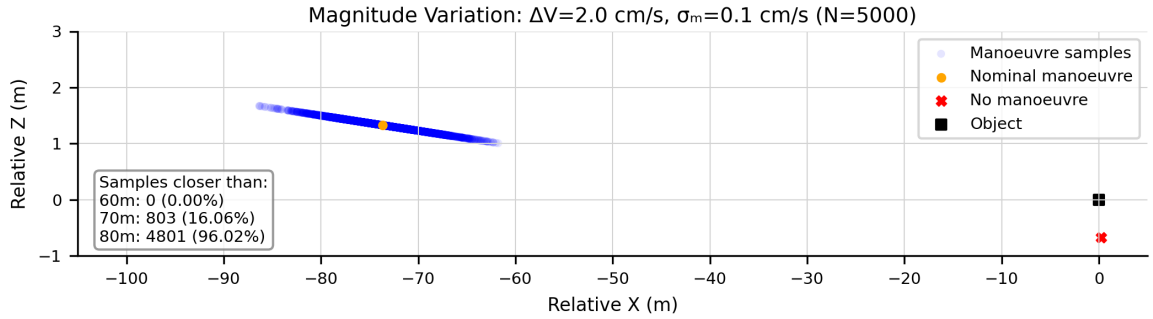
4.3.1 ΔV magnitude variation scenario

Manoeuvre samples presented in both figures 4a and 4b correspond to the cases where the standard deviation of the ΔV magnitude (σ_m) is equal to $0.05\,\mathrm{cm/s}$ and $0.1\,\mathrm{cm/s}$, respectively.

For $\sigma_m = 0.05\,\mathrm{cm/s}$, the samples are more concentrated around the nominal value. Only 2.48% of these samples had an approach distance of less than $70\,\mathrm{m}$. The scenario with $\sigma_m = 0.1\,\mathrm{cm/s}$ yielded a worst-case distance of closest approach of approximately $62\,\mathrm{m}$, with 16.06% of the samples having an approach distance closer than $70\,\mathrm{m}$. In both cases, all samples lie on a line that, when extended, passes through the *No manoeuvre* point, as expected for a perfectly aligned retrograde burn.



(a) Position distribution for ΔV magnitude variation with $\sigma_m = 0.05\,\mathrm{cm/s}$.



(b) Position distribution for ΔV magnitude variation with $\sigma_m = 0.1\,\mathrm{cm/s}$.

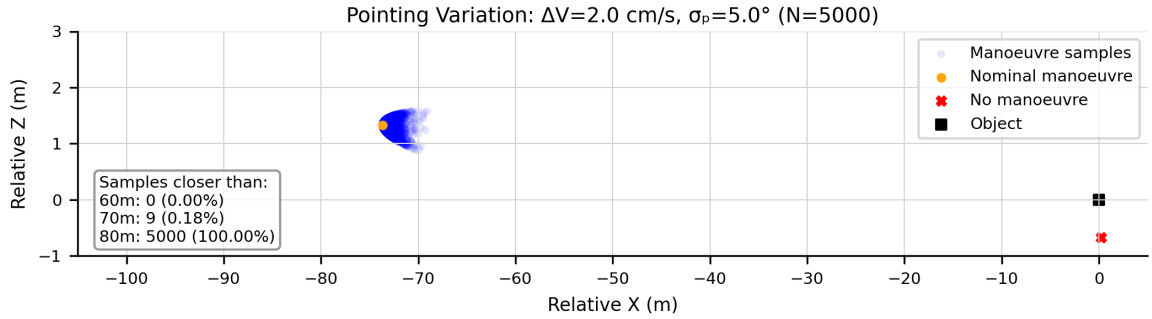
Figure 4. Relative position distribution at the time of closest approach with varying ΔV magnitude

4.3.2 Pointing error variation scenario

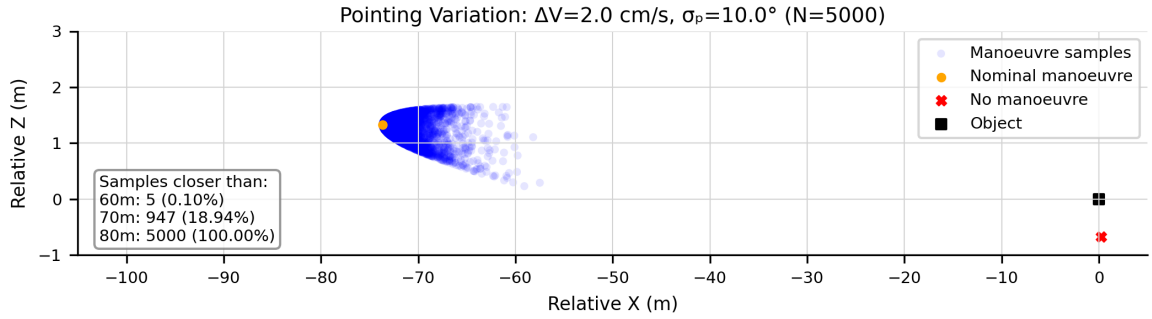
The pointing error scenario was analysed for three different values of the pointing error distribution's standard deviation: $\sigma_p = 5°$, $\sigma_p = 10°$, and $\sigma_p = 15°$. The results are shown in Figures 5a, 5b, and 5c, respectively.

Intuitively, results for greater pointing errors show a greater spread and a smaller distance at the time of closest approach. The worst-case closest approach was approximately $70\,\mathrm{m}$ for the scenario with $\sigma_p = 5°$, approximately $60\,\mathrm{m}$ for the scenario with $\sigma_p = 10°$, and approximately $50\,\mathrm{m}$ for the

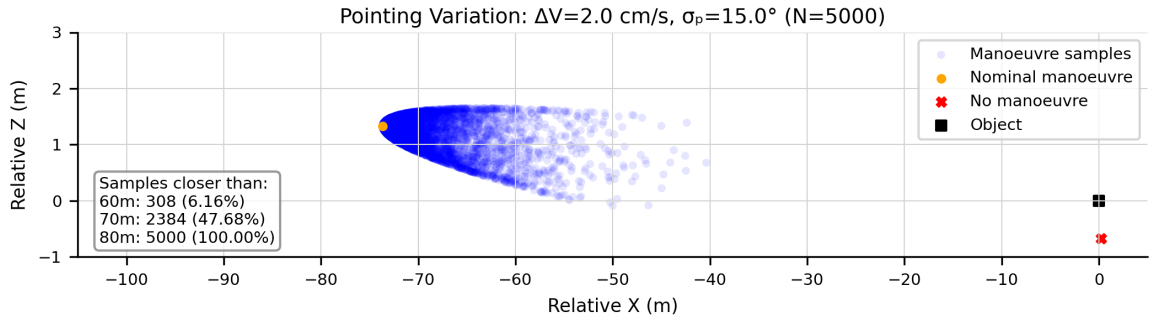
scenario with $\sigma_p = 15°$. In the three cases, 0.18%, 18.94%, and 47.68% of the samples had an approach distance of less than $70\,\mathrm{m}$, respectively. The distribution of points representing samples at the time of closest approach creates a characteristic 'parabola' shape, with many points concentrated along the set's boundary.



(a) Position distribution for pointing error with $\sigma_p = 5°$



(b) Position distribution for pointing error with $\sigma_p = 10°$



(c) Position distribution for pointing error with $\sigma_p = 15°$

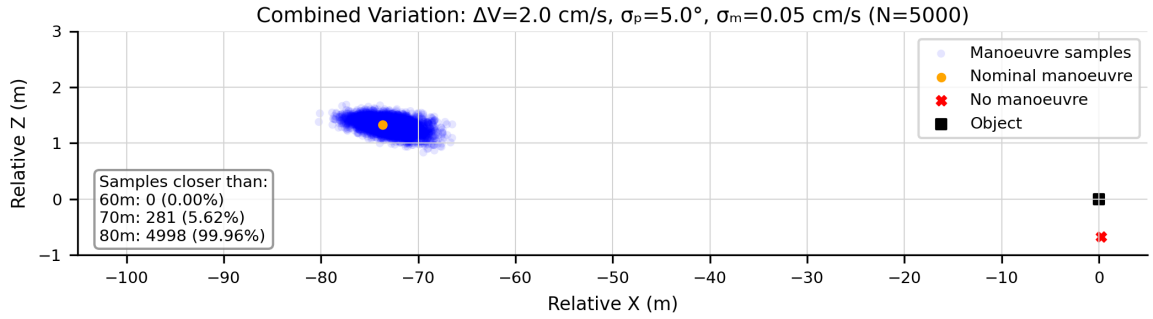
Figure 5. Relative position distribution at the time of closest approach with varying pointing errors

4.3.3 Combined scenario

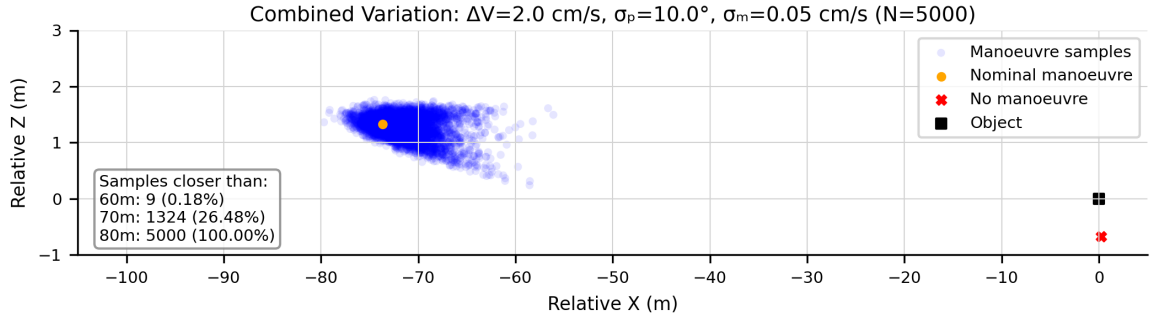
The combined scenario includes both sources of inaccuracies, with $\sigma_p = 5°$ and $\sigma_p = 10°$ for the pointing error, and $\sigma_m = 0.05\,\mathrm{cm/s}$ and $\sigma_m = 0.1\,\mathrm{cm/s}$ for the ΔV magnitude. The results are shown in Figure 6.

The results for the scenario with $\sigma_p = 5°$ and $\sigma_m = 0.05\,\mathrm{cm/s}$ are concentrated around $75\,\mathrm{m}$ in the X direction, with 5.62% of the samples within $70\,\mathrm{m}$ of the object. For $\sigma_p = 10°$ and $\sigma_m = 0.05\,\mathrm{cm/s}$, the samples are more spread out and closer to the object – 26.48% are closer than $70\,\mathrm{m}$, and 0.18% are closer than $60\,\mathrm{m}$.

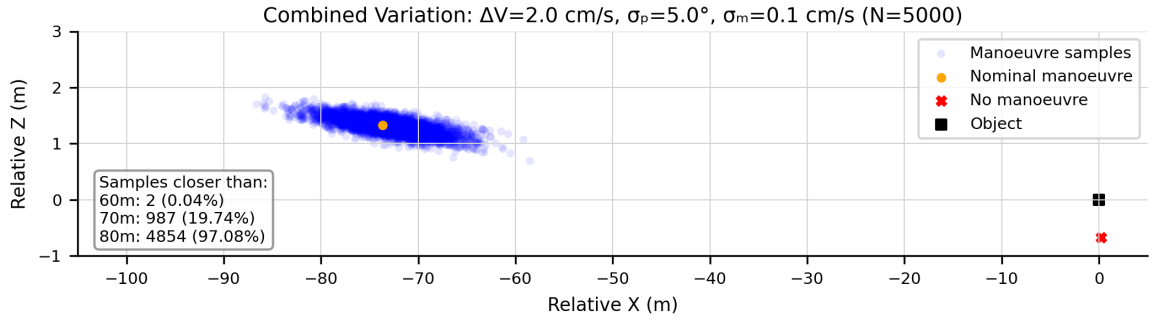
For $\sigma_p = 5°$ and $\sigma_m = 0.1\,\mathrm{cm/s}$, the samples are more spread out in the X direction than in the previous cases, with 19.74% of the samples within $70\,\mathrm{m}$ and 0.04% within $60\,\mathrm{m}$ of the object. Finally,



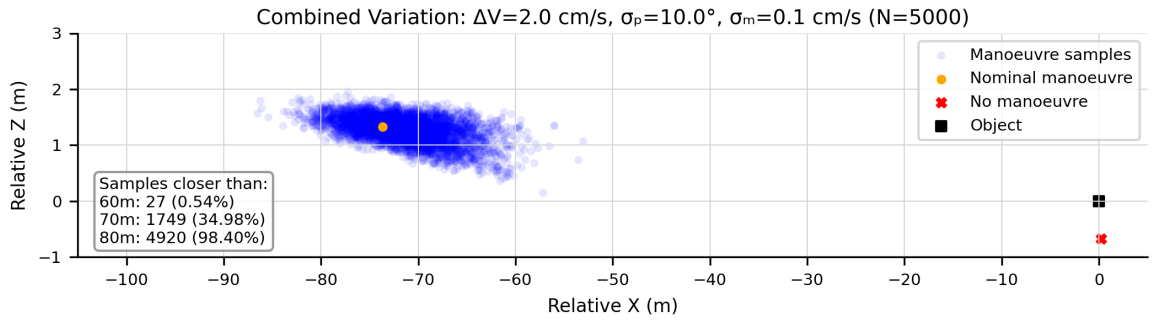
(a) Combined scenario with $\sigma_p = 5°$ and $\sigma_m = 0.05\,\mathrm{cm/s}$.



(b) Combined scenario with $\sigma_p = 10°$ and $\sigma_m = 0.05\,\mathrm{cm/s}$.



(c) Combined scenario with $\sigma_p = 5°$ and $\sigma_m = 0.1\,\mathrm{cm/s}$.



(d) Combined scenario with $\sigma_p = 10°$ and $\sigma_m = 0.1\,\mathrm{cm/s}$.

Figure 6. Relative position distribution at the time of closest approach with varying pointing error and ΔV magnitude

for $\sigma_p = 10°$ and $\sigma_m = 0.1\,\mathrm{cm/s}$, the samples are most spread out and closest to the object, with 34.98% being closer than $70\,\mathrm{m}$, and 0.54% being closer than $60\,\mathrm{m}$.

Table 1 presents the number of samples with the distance of closest approach below $70\,\mathrm{m}$ for different values of σ_p and σ_m. With $\sigma_p = 5°$ and $\sigma_p = 10°$, the doubling of σ_m results in increases of the total number of samples closer than $70\,\mathrm{m}$ by factors of 3.51 and 1.32, respectively. With $\sigma_m = 0.05\,\mathrm{cm/s}$ and $\sigma_m = 0.1\,\mathrm{cm/s}$, the doubling of σ_p results in increases by factors of 4.71 and 1.77.

Table 1. Number of samples with the closest approach below 70 m for different values of σ_p and σ_m

$\sigma_p \backslash \sigma_m$	**0.05 cm/s**	**0.1 cm/s**	**Change**
5°	281	987	x3.51
10°	1324	1749	x1.32
Change	x4.71	x1.77	

5 DISCUSSION

The main goals of this paper were to examine the developed Open GPU-Parallelised Orbit Propagator (OpenGPOP) and to demonstrate its application in the analysis of collision avoidance capabilities of the PW-Sat3 satellite under the influence of AOCS inaccuracies.

Compared to the conventional serial simulation approach, the proposed tool enables significantly faster orbit propagation for events involving a large number of objects, thereby improving scalability. Within the time frames required for Orekit to propagate a single path using the RK4 method, OpenGPOP can propagate thousands of independent orbits. At the same time, the results remain in close agreement with the reference solutions and real-world data, with minor inconsistencies likely stemming from the use of an older, JGM-3 gravity model.

The application to the PW-Sat3 case study demonstrated not only the practical utility of the tool but also provided insights into the impact of imprecision in the AOCS on the performance of collision avoidance manoeuvres. The results indicate that variations in both pointing and ΔV magnitude can measurably affect the distance of closest approach, with pointing errors appearing to have a more pronounced effect. Nevertheless, across all test cases, the vast majority of samples still had the distance of closest approach greater than $60\,\mathrm{m}$, suggesting that the proposed AOCS design should provide sufficient collision avoidance capabilities for PW-Sat3, even in the presence of inaccuracies.

Although OpenGPOP development is still in progress, the presented results provide a strong foundation for further application in areas that require efficient, scalable computation, such as modelling swarms and fragmentation events. Currently, the satellite constellations reach numbers of objects as high as several thousand (over 10,000 for Starlink as of March 2026), and large fragmentation events can involve thousands of elements. Such a scale of considered systems presents an opportunity for GPU acceleration, and the support for dividing the propagated objects into subsets for conjunction checking, featured in OpenGPOP, can help limit the computational burden. For instance, in the case of a fragmentation event, the debris can be placed in one set, and all other objects in another set, so that conjunction checks are performed only between the debris and the other objects, but not between the debris themselves.

6 CONCLUSION AND FUTURE DEVELOPMENT

In conclusion, the Open GPU-Parallelised Orbit Propagator, although still under development, already proves to be a useful mission analysis tool. As shown by the analysis of PW-Sat3's manoeuvre inaccuracies, Monte Carlo orbit propagation simulations supported by parallelised computations can quickly yield comprehensive, accurate results. In the future, the team plans to extend the tool to include additional acceleration source models, such as atmospheric drag, support for covariance matrix propagation, and a dedicated Python scripting interface. Further optimisations of computations are also planned.

ACKNOWLEDGEMENTS

This research was carried out with the support of the High Performance Computing Center at Faculty of Mathematics and Information Science, Warsaw University of Technology.

APPENDIX A SOURCE CODE

The source code of Open GPU-Parallelised Orbit Propagator is available under the GNU General Public License v3.0 at:

`https://github.com/OpenGPOP/OpenGPOP`

REFERENCES

[1] Matthew M. Berry and Liam M. Healy. Implementation of Gauss-Jackson integration for orbit propagation. *The Journal of the Astronautical Sciences*, 52(3):331–357, 2004.

[2] NVIDIA Corporation. cuBLAS: GPU-accelerated basic linear algebra subroutines. `https://developer.nvidia.com/cublas`, 2026. Accessed: 2026-03-24.

[3] Robert G Gottlieb. Fast gravity, gravity partials, normalized gravity, gravity gradient torque and magnetic field: derivation, code and data. Technical report, 1993.

[4] Gérard Petit and Brian Luzum. IERS conventions (2010). Technical report, 2010.

[5] NVIDIA Corporation. Thrust: The C++ parallel algorithms library. `https://developer.nvidia.com/thrust`, 2026. Accessed: 2026-03-24.

[6] IAU Standards of Fundamental Astronomy (SOFA). `https://www.iausofa.org/`, 2026. Accessed: 2026-03-22.

[7] Randy A Eckman, Aaron J Brown, Daniel R Adamo, and Robert G Gottlieb. Normalization and implementation of three gravitational acceleration models. Technical report, 2016.

[8] European Space Agency and Copernicus Precise Orbit Determination Service. Copernicus Sentinel-1A precise orbit ephemerides (AUX_POEORB). Copernicus Data Space Ecosystem, 2023.

[9] maisonobe, Bryan Cazabonne, Evan Ward, Romain Serra, Maxime Journot, Sébastien Dinot, Guilhem Bonnefille, Thomas Neidhart, Luc Maisonobe, Clément Jonglez, Mark Rutten, yjeand, Julio Hernanz, lirw1984, Vincent Cucchietti, Andrew Goetz, Guiuux, gaetanpierre0, Lars Næsbye Christensen, Alberto Fossà, jvalet, Alberto Ferrero, gabb5, liscju, Christopher Schank, Tanner Mills, plan3d, Vyom Yadav, Shiva Iyer, and Petrus Hyvönen. Cs-si/orekit: 12.2, October 2024.